

Manual EntityGuard

EntityGuard — Reliable Actions in Home Assistant

Many devices in Home Assistant do not always respond immediately. A light turns on, but Home Assistant only notices it later. A switch receives a command but doesn't execute it. A robot vacuum is temporarily offline. A Zigbee device is slow. A WiFi device occasionally misses a signal.

The result?

- automations think something failed
- actions are not retried
- devices remain in the wrong state
- flows become unreliable

That's why I created **EntityGuard**.

What does EntityGuard do?

EntityGuard is a smart Home Assistant script that ensures a device does what you ask — even when the device:

- is slow
- responds poorly
- reports its state too late
- occasionally misses a command
- is temporarily offline

EntityGuard:

- executes the action
- waits until the state is updated
- checks whether the state is correct
- retries if necessary
- performs multiple attempts
- uses configurable delays
- uses intelligent backoff strategies
- stops as soon as it succeeds
- or provides a clear error message if it fails

In short: **EntityGuard is a reliable executor of Home Assistant actions.**

Who is EntityGuard for?

Use EntityGuard when:

- devices sometimes report their state too late
- actions occasionally don't arrive
- you want to be absolutely sure something is truly on/off/open/closed
- you want safe toggle actions
- you want reliable automations
- you use Zigbee devices
- you use WiFi devices
- you use robot vacuums
- you use covers
- you use smart locks

Manual EntityGuard

In short:

EntityGuard ensures Home Assistant doesn't give up when a device acts stubborn.

Why EntityGuard is so useful

EntityGuard:

- handles temporary failures
- handles slow state updates
- handles missed commands
- makes automations extremely reliable
- works with all major device types
- has clear parameters
- has smart retry strategies
- has notifications
- has optional logging
- is generic and usable everywhere

How EntityGuard works internally

EntityGuard operates in four major phases:

1. **Validation Phase**
2. **Retry Phase**
3. **Success Phase**
4. **Failure Phase**

Below is a detailed explanation of each phase.

Validation Phase

Before EntityGuard does anything, it checks:

Does the entity exist?

If Home Assistant reports:

- unknown
- unavailable

EntityGuard stops immediately.

Is the action valid for this device?

Examples:

- switch.turn_on → valid
- switch.open_cover → invalid
- vacuum.media_play → invalid

If the action does not match the domain, EntityGuard stops immediately.

Manual EntityGuard

Is it a scene?

Scenes do not have a state. Therefore:

- EntityGuard executes the scene
- sends an event
- stops immediately
- no retries
- no state checking

Toggle logic

For toggle actions, EntityGuard checks:

- device is **on** → expected state = **off**
- device is **off** → expected state = **on**
- device is in a weird state → script stops safely

Retry Phase

EntityGuard keeps trying until it succeeds — or until all attempts are used.

Step 1 — Send the action

Examples:

- light.turn_on
- switch.turn_off
- cover.open_cover
- vacuum.start

Step 2 — Wait (state_delay)

Some devices perform the action, but Home Assistant only sees it later.

EntityGuard waits:

0–3 seconds (depending on the domain)

so the state has time to update.

Step 3 — Check the state

EntityGuard checks:

“Is the state now what it should be?”

Example:

You call light.turn_on

Expected state = on

EntityGuard checks: *is the light really on?*

Step 4 — Wait for state (timeout)

If the state is still incorrect, EntityGuard waits until:

- the correct state arrives
- or the timeout expires

Manual EntityGuard

Step 5 — Retry

If the state is still incorrect:

- EntityGuard tries again
- waits between attempts
- uses the selected backoff strategy:

constant

5 → 5 → 5 → 5 → 5 sec

linear

5 → 10 → 15 → 20 → 25 sec

exponential

5 → 10 → 20 → 40 → 80 sec

Step 6 — Repeat until success or until attempts are exhausted

Success Phase

As soon as the state is correct:

- EntityGuard stops
- sends an event with all details
- logs a clean trace
- performs no additional retries

Failure Phase

If all retries are used:

- EntityGuard sends an event with all details
- sends a notification (if notify=true)
- stops with a clear error message
- automations can use this to trigger follow-up actions

What EntityGuard calculates internally

EntityGuard automatically determines:

- **domain** (light, switch, fan, cover, etc.)
- **action** (turn_on, turn_off, open_cover, etc.)
- **current state** states(entity_id)
- **supported actions per domain**
- **expected end state** based on the action and domain

Manual EntityGuard

EntityGuard — Supported Actions per Domain

EntityGuard supports a wide range of Home Assistant domains. Below is the complete list of supported actions, including the expected end state that EntityGuard uses internally.

This list is essential because EntityGuard only works with actions that:

- have a clear, verifiable end state
- belong to a domain that reports a state
- can be checked reliably
- can be retried safely

Scenes are the only exception — they have no state and are handled separately.

Lights / Switches / Fans / Groups / Input Boolean

These domains all share the same end-state logic.

Supported Actions

- turn_on → expected state: **on**
- turn_off → expected state: **off**
- toggle → expected state: depends on current state

Toggle logic

EntityGuard determines the expected state based on the current state:

- current state = on → expected = off
- current state = off → expected = on
- current state is anything else → script stops safely

Covers

Supported Actions

- open_cover → expected state: **open**
- close_cover → expected state: **closed**
- stop_cover → expected state: **stopped**

Covers often report their final state with a delay, which makes state_delay important.

Locks

Supported Actions

- lock → expected state: **locked**
- unlock → expected state: **unlocked**

Smart locks may briefly report transitional states; EntityGuard waits until the final state is received.

Manual EntityGuard

Vacuum (Robot Vacuums)

Supported Actions

- start → expected state: **cleaning**
- pause → expected state: **paused**
- return_to_base → expected state: **returning**

Notes

Some vacuums support additional actions like stop or clean_spot. These are not included in the script by default but can be added later.

Robot vacuums often have:

- WiFi delays
- docking delays
- slow state updates

This makes state_delay especially important.

Media Player

Supported Actions

- turn_on → expected state: **on**
- turn_off → expected state: **off**
- media_play → expected state: **playing**
- media_pause → expected state: **paused**
- media_stop → expected state: **idle**

Media players often report transitional states such as buffering or loading. EntityGuard waits until the final state matches the expected one.

Scenes (Special Handling)

Scenes do not have a state.

Supported Action

- scene.turn_on

Special Behavior

EntityGuard:

- executes the scene
- sends an event
- stops immediately
- performs **no retries**
- performs **no state checking**

Scenes are always treated as successful unless the action itself fails.

Manual EntityGuard

Climate (Thermostats, HVAC, Heat Pumps)

Supported Actions

- `turn_on` → expected state: **on**
- `turn_off` → expected state: **off**
- `set_hvac_mode` → expected state: value of `data.hvac_mode`

Important

The script uses:

```
{{ data.hvac_mode | default('unknown') }}
```

This means:

- You must provide `hvac_mode` in the data: block
- If missing, the expected state becomes unknown → script stops safely

Climate devices are often slow to update their state, so `state_delay` is crucial.

Humidifier / Dehumidifier

Supported Actions

- `turn_on` → expected state: **on**
- `turn_off` → expected state: **off**

These devices behave similarly to switches and lights.

Water Heater

Supported Actions

- `turn_on` → expected state: **on**
- `turn_off` → expected state: **off**

Water heaters often have slow state updates; consider using a higher `state_delay`.

Alarm Control Panel

Supported Actions

- `arm_away` → expected state: **armed_away**
- `arm_home` → expected state: **armed_home**
- `arm_night` → expected state: **armed_night**
- `disarm` → expected state: **disarmed**

Alarm systems may briefly report transitional states such as arming. EntityGuard waits until the final state matches the expected one.

Manual EntityGuard

What happens when an action is not supported?

EntityGuard:

1. calculates the expected state
2. if the expected state is unknown → the action is invalid
3. script stops immediately
4. a clear error message is shown
5. the list of supported domains is displayed

This prevents:

- invalid actions
- unsafe toggles
- domain mismatches
- unpredictable behavior

Why this list matters

This list ensures that EntityGuard:

- only executes actions that can be verified
- only works with domains that report a clear state
- handles scenes safely
- handles toggle actions correctly
- handles climate devices correctly
- handles robot vacuums correctly
- handles alarm systems correctly

It also ensures:

- predictable behavior
- reliable automations
- safe retries
- correct end-state validation

Manual EntityGuard

EntityGuard — Required and Optional Parameters

EntityGuard uses a set of parameters that allow you to control exactly how actions are executed. Three parameters are **required**, and the rest are **optional**. If you omit optional parameters, EntityGuard automatically uses safe defaults.

Required Parameters

These three parameters must always be provided. Without them, EntityGuard cannot operate.

1. description

A short description of what you are doing.

Examples:

- “Living room light on”
- “Close shutter”
- “Ventilation high”

This description is used in:

- logging
- notifications
- trace output

It has **no effect** on how the script behaves.

2. entity_id

The entity you want to control.

Examples:

- light.kitchen
- switch.lamp_1_socket_1
- cover.livingroom_shutter
- vacuum.roborock_s7

EntityGuard checks:

- does the entity exist?
- is the entity available?

If not → the script stops immediately.

Manual EntityGuard

3. action

The Home Assistant action that should be executed.

Examples:

- light.turn_on
- switch.turn_off
- cover.open_cover

EntityGuard automatically splits this into:

- **domain** → light
- **action name** → turn_on

Then it determines:

- which end state belongs to this action
- whether the action is valid for this domain

If the action is invalid → the script stops safely.

Optional Parameters

These parameters can be omitted — EntityGuard will fill them with safe defaults.

4. data

Extra settings for the action.

Examples:

- brightness
- color
- speed
- mode
- hvac_mode
- transition

Example:

data:

brightness_pct: 50

If you do not provide data, EntityGuard uses an empty dictionary {}.

5. retries

How many times EntityGuard should try the action.

Default: 3 attempts

Typical usage:

- Zigbee → 3–5
- WiFi → 2–3
- robot vacuums → 1–2
- covers → 1–3

Manual EntityGuard

6. timeout

How long EntityGuard waits for the correct state.

Default: 15 seconds

If the state arrives too late → timeout → new attempt.

7. retry_delay

Delay between attempts.

Default: 5 seconds

Typical usage:

- Zigbee → 5–10 sec
- WiFi → 3–5 sec
- robot vacuums → 10–20 sec

8. backoff_mode

How the delay increases with each retry.

Available modes:

constant

Every attempt uses the same delay. Example: 5 → 5 → 5 → 5 sec

linear

Delay increases slowly. Example: 5 → 10 → 15 → 20 sec

exponential

Delay increases quickly. Example: 5 → 10 → 20 → 40 sec

Default: linear

9. state_delay

How long EntityGuard waits before checking the state.

This parameter is extremely important.

Why?

Many devices perform the action immediately, but Home Assistant only sees the updated state later.

Example:

- 0.0 sec → light.turn_on sent
- 0.2 sec → light turns on
- 0.8 sec → Home Assistant reports “on”

If EntityGuard checks too early, it thinks:

“Light is still off → action failed.”

But that’s not true — the state simply arrived late.

Default: 2 seconds

What is state_delay?

state_delay is a waiting period before EntityGuard checks whether a device is truly on/off/open/closed.

Manual EntityGuard

Why does this exist?

Because many devices **do** what you ask, but Home Assistant only sees the updated state later.

Example:

- 0.0 sec → command sent
- 0.2 sec → device reacts
- 0.8 sec → state update arrives

If EntityGuard checks at 0.1 sec, it thinks the action failed.

10. notify

Send a notification if all attempts fail.

Default: true

Set to false for silent automations.

11. notify_target

The notify service where messages should be sent.

Examples:

- notify.henk
- notify.mobile_app_iphone_van_henk
- notify.telegram_henk

Default: notify.henk

This allows you to choose who receives failure messages.

Manual EntityGuard

EntityGuard — Understanding State Delay

state_delay is a waiting period that EntityGuard uses **before** checking whether a device is truly on/off/open/closed.

It is one of the most important parameters in the entire script.

Why does state_delay exist?

Because many devices **do** what you ask, but Home Assistant only sees the updated state **later**.

Example:

- **0.0 sec** → light.turn_on sent
- **0.2 sec** → the light turns on
- **0.8 sec** → Home Assistant finally reports “on”

If EntityGuard checks the state at **0.1 sec**, it thinks:

“The light is still off → action failed.”

But that’s not true — the state simply arrived late.

state_delay prevents false failures by giving the device time to update its state.

Simple explanation

Imagine telling a child:

“Turn on the light.”

The child walks to the switch. You look **before** the child reaches the switch. You think:

“He’s not doing it!”

But you simply looked too early.

state_delay makes EntityGuard wait long enough for the device to respond.

Typical state_delay values

Different device types report their state at different speeds.

Switches

Fast devices → state_delay: 1

Lights (Zigbee, WiFi, Hue, Ikea, etc.)

Often slow → state_delay: 2 Sometimes even 3 for Zigbee

Fans

Moderate → state_delay: 2

Groups

Multiple devices → slower updates → state_delay: 3

Covers (shutters, blinds, screens)

State updates often delayed → state_delay: 1

Locks

Short delay → state_delay: 1

Vacuum cleaners (robot vacuums)

WiFi delays, docking delays → state_delay: 2

Media players

Short delay → state_delay: 1

Manual EntityGuard

Climate devices (thermostats, HVAC, heat pumps)

Always slow → state_delay: 3

Humidifier / Dehumidifier

Fast → state_delay: 1–2

Water heater

Slow → state_delay: 2–3

Why state_delay prevents false failures

Without state_delay:

- the device reacts
- but Home Assistant hasn't updated yet
- EntityGuard thinks the action failed
- retries start unnecessarily
- notifications may be sent incorrectly

With state_delay:

- the device reacts
- EntityGuard waits
- Home Assistant updates
- EntityGuard sees the correct state
- the script stops successfully

This makes automations **far more reliable**.

Examples per domain

Switch

```
- action: script.entityguard
  data:
    description: "Living room light on"
    entity_id: switch.livingroom_light
    action: switch.turn_on
    state_delay: 1
```

Input Boolean

```
- action: script.entityguard
  data:
    description: "Vacation mode on"
    entity_id: input_boolean.vacation_mode
    action: input_boolean.turn_on
    state_delay: 0
```

Manual EntityGuard

Light (with brightness)

```
- action: script.entityguard
  data:
    description: "Kitchen light on"
    entity_id: light.kitchen
    action: light.turn_on
    data:
      brightness_pct: 60
    state_delay: 2
```

Fan

```
- action: script.entityguard
  data:
    description: "Fan high"
    entity_id: fan.livingroom
    action: fan.turn_on
    state_delay: 2
```

Group

```
- action: script.entityguard
  data:
    description: "Living room group on"
    entity_id: group.livingroom
    action: homeassistant.turn_on
    state_delay: 3
```

Cover

```
- action: script.entityguard
  data:
    description: "Open shutter"
    entity_id: cover.livingroom_shutter
    action: cover.open_cover
    state_delay: 1
```

Lock

```
- action: script.entityguard
  data:
    description: "Front door locked"
    entity_id: lock.frontdoor
    action: lock.lock
    state_delay: 1
```

Manual EntityGuard

Vacuum

```
- action: script.entityguard
  data:
    description: "Start Roborock"
    entity_id: vacuum.roborock_s7
    action: vacuum.start
    state_delay: 2
```

Media Player

```
- action: script.entityguard
  data:
    description: "Play media"
    entity_id: media_player.livingroom
    action: media_player.media_play
    state_delay: 1
```

Climate

```
- action: script.entityguard
  data:
    description: "Heating on"
    entity_id: climate.livingroom
    action: climate.set_hvac_mode
    data:
      hvac_mode: heat
    state_delay: 3
```


Manual EntityGuard

EntityGuard — Example Automations

Below you will find all example automations translated into English. They follow the same structure as your original Dutch versions, but are written in clear, natural English suitable for international users.

Switch Automation

```
- id: livingroom_light_on
alias: Living Room Light On via EntityGuard
triggers:
  - trigger: state
    entity_id: input_boolean.test
    to: "on"
action:
  - action: script.entityguard
data:
  description: "Living room light on"
  entity_id: switch.livingroom_light
  action: switch.turn_on
  retries: 3
  timeout: 15
  retry_delay: 10
  backoff_mode: linear
  state_delay: 1
  notify: true
  notify_target: notify.mobile_app_klaas
mode: single
```

What this does

- The input_boolean turns on
- EntityGuard tries to turn on the light
- It checks the state
- Retries if necessary
- Sends a notification if it fails

Manual EntityGuard

Toggle Automation

- id: livingroom_light_toggle
alias: Living Room Light Toggle via EntityGuard
triggers:
- trigger: state
entity_id: input_boolean.toggle_light
to: "on"
action:
- action: script.entityguard
data:
description: "Toggle living room light"
entity_id: switch.livingroom_light
action: switch.toggle
state_delay: 1
mode: single

What this does

EntityGuard checks:

- light is **on** → it must turn **off**
- light is **off** → it must turn **on**
- light is in a weird state → script stops safely

Zigbee Light with Brightness

- id: kitchen_light_brightness
alias: Kitchen Light On with Brightness
triggers:
- trigger: state
entity_id: input_boolean.kitchen_on
to: "on"
action:
- action: script.entityguard
data:
description: "Kitchen light on"
entity_id: light.kitchen
action: light.turn_on
data:
brightness_pct: "{{ states('input_number.kitchen_brightness') | int }}"
retries: 3
timeout: 15
retry_delay: 10
backoff_mode: linear
state_delay: 2
notify: false
mode: single

Why state_delay = 2?

Zigbee lights often report their state late.

Manual EntityGuard

Cover (Shutter / Blind / Screen)

Open

- id: livingroom_shutter_open
alias: Living Room Shutter Open
triggers:
- trigger: state
entity_id: input_boolean.shutter_open
to: "on"
action:
- action: script.entityguard
data:
description: "Living room shutter open"
entity_id: cover.livingroom_shutter
action: cover.open_cover
state_delay: 1
mode: single

Close

- id: livingroom_shutter_close
alias: Living Room Shutter Close
triggers:
- trigger: state
entity_id: input_boolean.shutter_close
to: "on"
action:
- action: script.entityguard
data:
description: "Living room shutter close"
entity_id: cover.livingroom_shutter
action: cover.close_cover
mode: single

Stop

- id: livingroom_shutter_stop
alias: Living Room Shutter Stop
triggers:
- trigger: state
entity_id: input_boolean.shutter_stop
to: "on"
action:
- action: script.entityguard
data:
description: "Living room shutter stop"
entity_id: cover.livingroom_shutter
action: cover.stop_cover
mode: single

Manual EntityGuard

Lock (Smart Lock)

Lock

- id: frontdoor_lock
alias: Front Door Lock
triggers:
- trigger: state
entity_id: input_boolean.lock_frontdoor
to: "on"
action:
- action: script.entityguard
data:
description: "Front door locked"
entity_id: lock.frontdoor
action: lock.lock
state_delay: 1
mode: single

Unlock

- id: frontdoor_unlock
alias: Front Door Unlock
triggers:
- trigger: state
entity_id: input_boolean.unlock_frontdoor
to: "on"
action:
- action: script.entityguard
data:
description: "Front door unlocked"
entity_id: lock.frontdoor
action: lock.unlock
mode: single

Manual EntityGuard

Vacuum (Robot Vacuum)

Start

- id: roborock_start
alias: Start Roborock
triggers:
- trigger: state
entity_id: input_boolean.cleaning
to: "on"
action:
- action: script.entityguard
data:
description: "Start Roborock"
entity_id: vacuum.roborock_s7
action: vacuum.start
state_delay: 2
mode: single

Pause

- id: roborock_pause
alias: Pause Roborock
triggers:
- trigger: state
entity_id: input_boolean.pause_cleaning
to: "on"
action:
- action: script.entityguard
data:
description: "Pause Roborock"
entity_id: vacuum.roborock_s7
action: vacuum.pause
mode: single

Return to Dock

- id: roborock_dock
alias: Roborock Return to Dock
triggers:
- trigger: state
entity_id: input_boolean.return_to_dock
to: "on"
action:
- action: script.entityguard
data:
description: "Roborock return to dock"
entity_id: vacuum.roborock_s7
action: vacuum.return_to_base
mode: single

Media Player

Manual EntityGuard

Play

- id: media_play
alias: Play Media
triggers:
- trigger: state
entity_id: input_boolean.media_play
to: "on"
action:
- action: script.entityguard
data:
description: "Play media"
entity_id: media_player.livingroom
action: media_player.media_play
state_delay: 1
mode: single

Pause

- id: media_pause
alias: Pause Media
triggers:
- trigger: state
entity_id: input_boolean.media_pause
to: "on"
action:
- action: script.entityguard
data:
description: "Pause media"
entity_id: media_player.livingroom
action: media_player.media_pause
mode: single

Stop

- id: media_stop
alias: Stop Media
triggers:
- trigger: state
entity_id: input_boolean.media_stop
to: "on"
action:
- action: script.entityguard
data:
description: "Stop media"
entity_id: media_player.livingroom
action: media_player.media_stop
mode: single

Manual EntityGuard

Scene (Special Handling)

Scenes have no state checking.

- id: evening_scene

alias: Activate Evening Scene

triggers:

- trigger: state

entity_id: input_boolean.evening

to: "on"

action:

- action: script.entityguard

data:

description: "Evening scene"

entity_id: scene.evening

action: scene.turn_on

mode: single

What EntityGuard does

- executes the scene
- sends an event
- stops immediately
- no retries
- no state checking

Climate (HVAC Mode)

- id: heating_on

alias: Living Room Heating On

triggers:

- trigger: state

entity_id: input_boolean.heating

to: "on"

action:

- action: script.entityguard

data:

description: "Heating on"

entity_id: climate.livingroom

action: climate.set_hvac_mode

data:

hvac_mode: heat

state_delay: 3

mode: single

Why state_delay = 3?

Climate devices are slow to update their state.

Manual EntityGuard

Alarm System

Arm Away

- id: alarm_arm_away
alias: Alarm Arm Away
triggers:
- trigger: state
entity_id: input_boolean.alarm_on
to: "on"
action:
- action: script.entityguard
data:
description: "Alarm arm away"
entity_id: alarm_control_panel.home
action: alarm_control_panel.arm_away
mode: single

Disarm

- id: alarm_disarm
alias: Alarm Disarm
triggers:
- trigger: state
entity_id: input_boolean.alarm_off
to: "on"
action:
- action: script.entityguard
data:
description: "Alarm disarm"
entity_id: alarm_control_panel.home
action: alarm_control_panel.disarm
mode: single

Manual EntityGuard

Automation with Exponential Backoff

```
- id: lamp_exponential_backoff
  alias: Lamp with Exponential Backoff
  triggers:
    - trigger: state
      entity_id: input_boolean.testlamp
      to: "on"
  action:
    - action: script.entityguard
      data:
        description: "Test lamp living room"
        entity_id: switch.testlamp
        action: switch.turn_on
        retries: 5
        timeout: 15
        retry_delay: 15
        backoff_mode: exponential
        notify: true
  mode: single
```

Manual EntityGuard

EntityGuard — Testing (Developer Tools)

You can test almost everything in EntityGuard using **Developer Tools** in Home Assistant. This allows you to:

- avoid creating automations
- immediately see what EntityGuard does
- see whether the state arrives too late
- verify retries
- check whether the action is valid
- check whether the entity exists
- test toggle logic
- test scene handling
- verify your settings
- learn how EntityGuard behaves internally

It is the fastest way to get familiar with EntityGuard.

Where to find it

In Home Assistant:

1. Go to **Developer Tools**
2. Click **Actions**
3. Choose **script.entityguard**
4. Fill in the data
5. Click **Perform Action**

That's all.

Step-by-Step Basic Test

Test: Turn on a light

Go to Developer Tools → Actions Choose script.entityguard Enter:

action: script.entityguard

data:

description: Test light

entity_id: light.window_light

action: light.turn_off

Click **Perform Action**.

What happens?

EntityGuard:

- executes the action
- waits for state_delay
- checks the state
- stops when successful
- shows everything in the trace

Manual EntityGuard

How to view the trace

1. Go to **Scripts**
2. Click **EntityGuard**
3. Click **Traces**
4. Select the latest trace

You will see:

- all steps
- all variables
- all retries
- all delays
- all error messages
- all events
- all timings

Your script stores multiple traces if you add:

trace:

stored_traces: 3

Test: Toggle

Toggle is tricky, but EntityGuard makes it safe.

Go to Developer Tools → Actions Choose script.entityguard Enter:

action: script.entityguard

data:

description: "Toggle test"

entity_id: light.kitchen

action: light.toggle

What happens?

EntityGuard checks:

- light is **on** → expected = **off**
- light is **off** → expected = **on**
- light is in a weird state → script stops safely

Test: Scene

Scenes have no state checking.

Go to Developer Tools → Actions Choose script.entityguard Enter:

action: script.entityguard

data:

description: "Scene test"

entity_id: scene.evening

action: scene.turn_on

Manual EntityGuard

What happens?

- scene is executed
- event is sent
- script stops immediately
- no retries
- no state checking

Test: Climate (HVAC Mode)

Go to Developer Tools → Actions Choose script.entityguard Enter:

action: script.entityguard

data:

description: "HVAC test"

entity_id: climate.livingroom

action: climate.set_hvac_mode

data:

hvac_mode: heat

What happens?

EntityGuard:

- executes the action
- waits state_delay (often 3 sec)
- checks whether climate is now "heat"
- retries if necessary

Test: Cover (Shutter)

action: script.entityguard

data:

description: "Shutter test"

entity_id: cover.livingroom_shutter

action: cover.open_cover

Test: Robot Vacuum

action: script.entityguard

data:

description: "Roborock test"

entity_id: vacuum.roborock_s7

action: vacuum.start

Manual EntityGuard

Test: Fake Entity (super useful!)

This is ideal for testing error handling.

Create a fake entity:

```
input_boolean.fake_test
```

Test:

```
action: script.entityguard
```

data:

```
description: "Fake test"
```

```
entity_id: input_boolean.fake_test
```

```
action: input_boolean.turn_on
```

What happens?

EntityGuard:

- executes the action
- state becomes “on” immediately
- script stops cleanly
- trace shows all details

You can use this to test:

- retries
- backoff
- error messages
- logging
- notifications

Test: Invalid Action

```
action: script.entityguard
```

data:

```
description: "Error test"
```

```
entity_id: light.kitchen
```

```
action: light.open_cover
```

What happens?

EntityGuard stops immediately with:

Unknown action. Check whether the action is valid for this domain.

And shows the list of supported domains.

Test: Entity Does Not Exist

```
action: script.entityguard
```

data:

```
description: "Error test"
```

```
entity_id: light.does_not_exist
```

```
action: light.turn_on
```

What happens?

EntityGuard stops with:

The specified entity does not exist or is unavailable.

Manual EntityGuard

Test: State Arrives Too Late

This is the most important test.

Use a Zigbee light.

Enter:

action: script.entityguard

data:

description: "Zigbee test"

entity_id: light.kitchen

action: light.turn_on

state_delay: 0

What happens?

- light turns on
- state arrives late
- EntityGuard thinks it failed
- retries start

Now try:

state_delay: 2

What happens?

- light turns on
- EntityGuard waits 2 seconds
- state arrives
- script stops successfully on attempt 1

Test: Backoff Modes

constant

retry_delay: 5

backoff_mode: constant

Delay stays the same:

5 → 5 → 5 → 5 → 5 sec

linear

retry_delay: 5

backoff_mode: linear

Delay increases slowly:

5 → 10 → 15 → 20 → 25 sec

exponential

retry_delay: 5

backoff_mode: exponential

Delay increases quickly:

5 → 10 → 20 → 40 → 80 sec

Manual EntityGuard

The trace shows:

- how long each retry waits
- how the delay increases
- how the backoff strategy behaves

EntityGuard — Troubleshooting

Below is the complete troubleshooting section, translated into clear English. Each problem includes:

- what is happening
- why it happens
- how to fix it
- what to adjust in EntityGuard

This section helps users diagnose issues quickly and understand how EntityGuard behaves.

1. “The light turns on, but EntityGuard says FAILED”

What is happening?

The light *does* turn on, but Home Assistant reports the updated state **too late**.

Why does this happen?

Zigbee and some WiFi lights often report their state **0.5–2 seconds later**.

Solution

Increase `state_delay`.

Example:

`state_delay: 2`

For Zigbee lights, sometimes even:

`state_delay: 3`

2. “Entity is unknown/unavailable”

What is happening?

The device is:

- offline
- not connected
- still waking up
- dropped from the Zigbee mesh
- lost WiFi
- has an empty battery

Solution

Check:

- device connection
- whether the entity exists
- whether the device is awake
- Zigbee/WiFi stability

Manual EntityGuard

If needed:

- restart the device
- restart Home Assistant

3. “Toggle doesn’t work correctly”

What is happening?

Toggle only works when the current state is **on** or **off**.

Some devices return unusual states:

- unknown
- unavailable
- idle
- paused
- cleaning

Solution

Use toggle only with:

- light
- switch
- input_boolean
- fan
- group
- homeassistant

4. “The script stops immediately”

What is happening?

The action is written incorrectly.

Examples:

- switch.turn.om
- light.turn_of
- cover.open
- vacuum.go_home
- media_player.play

Solution

Correct the action name.

5. “No notification received”

What is happening?

You set:

notify: false

Solution

Enable notifications:

notify: true

Manual EntityGuard

6. “Scene doesn’t behave as expected”

What is happening?

Scenes have **no state**.

EntityGuard behavior

- executes the scene
- sends an event
- stops immediately
- no retries
- no state checking

Solution

This is normal — scenes cannot be verified.

7. “Climate responds slowly”

What is happening?

Climate devices (HVAC, thermostats, heat pumps) are **always slow** with state updates.

Solution

Use:

state_delay: 3

8. “Vacuum doesn’t start immediately”

What is happening?

Robot vacuums often have:

- WiFi delays
- docking delays
- slow state updates

Solution

Use:

state_delay: 2

9. “Cover stays on ‘opening’ or ‘closing’”

What is happening?

Covers sometimes report their final state late.

Solution

Use:

state_delay: 1

10. “Backoff doesn’t behave as expected”

What is happening?

You expected linear backoff, but you set exponential.

Solution

Use:

backoff_mode: linear

Manual EntityGuard

11. “EntityGuard retries even though the action succeeded”

What is happening?

The device reacted, but the state arrived **too late**.

Solution

Increase:

state_delay

12. “EntityGuard stops without retries”

What is happening?

The state was correct immediately.

Solution

Nothing — this is normal.

EntityGuard — Conclusion

EntityGuard is a smart, reliable, and generic Home Assistant script designed to ensure that devices do exactly what you ask — even when they are slow, report their state late, or occasionally miss a command.

The script:

- supports all major domains
- includes safe toggle logic
- performs multiple attempts
- waits patiently for the correct state
- uses backoff strategies
- sends notifications on failure
- logs events with full details
- works with all types of devices
- has a clear API
- is easy to extend
- is easy to test via Developer Tools → Actions
- provides clear error messages
- provides clear trace logging
- is generic and usable everywhere

Manual EntityGuard

Thanks to its parameters, you can configure the script exactly the way you want:

- fast
- slow
- silent
- with notifications
- with retries
- with exponential backoff
- with brightness
- with hvac_mode
- with scenes
- with covers
- with robot vacuums

The script is designed with one goal:

Make Home Assistant reliable — no matter how stubborn your devices are.

What started as an idea to make Zigbee lights more reliable grew into a fully-fledged software component with:

- an internal architecture
- error handling
- logging
- documentation
- a clear API
- a generic design

EntityGuard is not a “YAML hack”, but a reusable Home Assistant library.

The script ensures that:

- automations become predictable
- devices respond reliably
- state delays are no longer a problem
- missed commands are automatically handled
- your smart home becomes truly smart

And the best part?

You can continue expanding the script with:

- new domains
- new actions
- new checks
- new logic

The design is fully prepared for this.

Manual EntityGuard

EntityGuard — Future Development

In the meantime, I will continue working on new ideas, improvements, and extensions for EntityGuard. The script is designed to grow, evolve, and adapt to new Home Assistant features, new device types, and new automation patterns.

Future updates may include:

- additional supported domains
- more advanced action validation
- extended end-state logic
- improved retry strategies
- deeper integration with Home Assistant's new automation engine
- optional debugging tools
- optional performance metrics
- expanded event data
- more detailed logging
- optional safety checks for complex devices
- community-requested features

EntityGuard is built as a flexible, modular foundation — a small but powerful library that can be expanded without breaking existing automations.

The goal remains the same:

Make Home Assistant reliable, predictable, and frustration-free — no matter how stubborn your devices are.

And with every update, EntityGuard becomes a little smarter, a little safer, and a little more capable.